Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2025/2026

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

RAPPORT DE STAGE PROFESSIONNEL

Spécialité: .....

Intitulé

.....

Lieu du stage **Réalisé par** **Encadré par**

Remerciements

Résumé

Ce rapport présente la conception et la réalisation, durant un stage professionnel d'un mois, d'une application web interne dédiée à la gestion des demandes techniques. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre V1 des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; intelligence artificielle.

Abstract

This report presents the design and implementation of an internal web application for managing technical requests. Developed during a one-month professional internship, the solution addresses a controlled internal scope. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the V1 scope from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Keywords : software engineering ; REST API ; Angular ; Spring Boot ; artificial intelligence.

Sommaire

Liste des figures **iv**

Liste des tableaux **v**

Liste des acronymes **vi**

Introduction générale **1**

Chapitre 1 — Contexte et cadrage du projet **2**
1.1 Organisme d'accueil et cadre du stage 2
1.2 Contexte, existant et problématique 3
1.3 Solution proposée, objectifs et périmètre 3
1.4 Méthodologie et planification du travail 4

Chapitre 2 — Analyse et conception **5**
2.1 Spécification des besoins 5
2.2 Conception fonctionnelle 7
2.3 Conception technique 8
2.4 Conception de la base de données 12

Chapitre 3 — Réalisation et validation **15**
3.1 Vue d'ensemble de la solution réalisée 15
3.2 Parcours complet d'une demande 15
3.3 Administration et contrôle des accès 18
3.4 Assistance IA et validation humaine 18
3.5 Tests et validation 19
3.6 Limites et perspectives 19

Conclusion générale **21**

Références **22**

Liste des figures

1 Logo de SAFYRON ENGINEERING 2

2 Diagramme global des cas d'utilisation de la V1 6
3 Enregistrement et qualification des demandes 8
4 Composants applicatifs et systèmes externes de la V1 10
5 Organisation des responsabilités dans la conception backend 11
6 Entités persistantes et cardinalités du modèle de données V1 12

7 Connexion à l'application 15
8 Création et qualification initiale d'une demande 16
9 Liste filtrable des demandes 16
10 Traitement d'une demande affectée 17
11 Clôture après examen de la résolution 17
12 Gestion des utilisateurs internes 18
13 Assistance IA lors de la qualification 19

Liste des tableaux

1 Planning prévisionnel du stage 4

2 Capacités fonctionnelles de la V1 7

3 Dictionnaire de données synthétique 14

4 Niveaux de validation et portée 20

Liste des acronymes

API	Interface de programmation d'application (*Application Programming Interface*).
DTO	Objet de transfert de données (*Data Transfer Object*).
HTTP	Protocole de transfert hypertexte (*Hypertext Transfer Protocol*).
IA	Intelligence artificielle.
JPA	Interface de persistance Java (*Java Persistence API*).
JWT	Jeton web JSON (*JSON Web Token*).
RBAC	Contrôle d'accès fondé sur les rôles (*Role-Based Access Control*).
REST	Transfert d'état représentationnel (*Representational State Transfer*).
UML	Langage de modélisation unifié (*Unified Modeling Language*).

Introduction générale

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING. Cette période d'un mois a été consacrée à un projet de génie logiciel lié au suivi des demandes techniques formulées par les clients de l'entreprise.

Dans les activités de l'entreprise, les demandes techniques formulées par les clients doivent être comprises, qualifiées et suivies jusqu'à l'achèvement de leur traitement. Chaque demande rassemble plusieurs informations utiles, notamment le client concerné, la nature du besoin, son niveau de priorité, la personne chargée du traitement et son état d'avancement. La continuité de ce suivi suppose que ces informations restent accessibles et cohérentes au fil des différentes étapes.

Le besoin général consiste ainsi à disposer d'un moyen commun pour centraliser les demandes, faciliter leur qualification, attribuer les responsabilités et conserver une trace de leur évolution. Il ne s'agit pas uniquement d'enregistrer une nouvelle demande, mais d'accompagner l'ensemble de son cycle de vie tout en maintenant un accès adapté aux responsabilités des utilisateurs internes.

La problématique retenue est donc la suivante : comment concevoir une solution interne cohérente permettant de centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et le contrôle humain des décisions importantes ?

Pour répondre à cette problématique, le projet propose une application web interne dédiée à la gestion des demandes techniques. Son objectif est de fournir un espace structuré pour leur prise en charge, depuis l'enregistrement jusqu'à la clôture. Une assistance fondée sur l'intelligence artificielle peut compléter la qualification, mais elle demeure facultative et ne remplace jamais la décision de l'utilisateur responsable.

Le rapport est organisé en trois chapitres. Le premier présente l'organisme d'accueil, le contexte, la problématique, les objectifs et la planification du stage. Le deuxième expose l'analyse des besoins et la conception de la solution. Le troisième décrit la réalisation, les mécanismes essentiels et les éléments de validation obtenus.

Chapitre 1

Contexte et cadrage du projet

Le cadre professionnel permet de situer l'origine du besoin et la problématique étudiée. La solution proposée, son périmètre et les objectifs retenus sont ensuite présentés, avant la démarche de travail et la planification du stage.

1.1 Organisme d'accueil et cadre du stage

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING, située rue Lac Toba, aux Berges du Lac à Tunis. L'entreprise exerce dans le domaine des études techniques industrielles. Ses activités comprennent notamment la protection incendie, les études de dangers et l'analyse des risques industriels.

La figure 1 présente l'identité visuelle de l'organisme d'accueil.



FIGURE 1 – Logo de SAFYRON ENGINEERING

Le stage professionnel d'un mois a été réalisé dans le cadre de la cinquième année du cycle d'ingénieur en génie logiciel à EPI Digital School Sousse. Le travail a été mené individuellement, conformément aux exigences de l'école, sans rattachement à un service particulier. L'encadrement professionnel a été assuré par M. Achref Ammar, fondateur de SAFYRON ENGINEERING.

L'observation du contexte de l'entreprise a permis de mieux comprendre la nature des études réalisées et la place des demandes formulées par les clients. Sur la base de cette compréhension des processus de travail, l'étudiant a proposé l'idée d'une application web interne de gestion et de suivi des demandes techniques.

1.2 Contexte, existant et problématique

Les activités de SAFYRON ENGINEERING conduisent à traiter des demandes techniques provenant de clients. Une demande doit être comprise, orientée vers la personne concernée et suivie jusqu'à l'achèvement de son traitement.

L'étude de l'existant porte sur la logique générale de traitement des demandes, sans présumer des outils utilisés auparavant. L'analyse du fonctionnement métier met en évidence plusieurs informations utiles au suivi : l'identité du client concerné, la nature du besoin, la priorité, la personne chargée du traitement, l'état courant et les changements successifs. Ces éléments doivent rester cohérents au cours de la prise en charge d'une demande.

Le besoin ne consiste donc pas seulement à enregistrer une demande. Le suivi doit permettre de la retrouver, de connaître son niveau d'avancement, d'identifier la personne responsable de son traitement et de conserver la trace des décisions importantes. Les intervenants disposent ainsi d'une vue claire de l'état de chaque demande et de la responsabilité associée à sa prise en charge.

La problématique du projet peut ainsi être formulée de la manière suivante : comment mettre à la disposition des intervenants internes un moyen cohérent pour centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et un accès adapté aux responsabilités de chacun ?

1.3 Solution proposée, objectifs et périmètre

Pour répondre à cette problématique, la solution proposée est une application web interne consacrée à la gestion et au suivi des demandes techniques. Elle fournit un espace commun dans lequel les demandes peuvent être enregistrées, qualifiées, affectées et suivies pendant leur traitement.

Les objectifs principaux sont de centraliser les demandes, de faciliter leur qualification et de formaliser leur cycle de vie. La solution doit également permettre d'affecter chaque demande, de suivre son traitement, de conserver un historique des changements et de protéger l'accès selon les responsabilités des utilisateurs. Une assistance fondée sur l'intelligence artificielle peut proposer une catégorie et une priorité ; elle reste facultative et toute proposition doit être validée par une personne avant son utilisation.

Les responsabilités internes se répartissent entre trois profils. Le Responsable technique pilote la prise en charge des demandes, l'Agent technique réalise leur traitement et l'Administrateur gère les comptes internes. Leurs interactions et leurs besoins sont détaillés dans l'analyse fonctionnelle.

Le périmètre retenu pour la V1 concerne les fonctions effectivement réalisées pour les utilisateurs internes. Le client y est représenté comme une entité métier associée à ses demandes, mais il ne dispose ni d'un compte ni d'un portail. L'assistance par intelligence artificielle ne

décide ni de la qualification ni de l'affectation : les choix finaux de catégorie et de priorité ainsi que les décisions de traitement restent sous contrôle humain. Ces limites correspondent à la durée du stage et distinguent la version réalisée des évolutions possibles.

1.4 Méthodologie et planification du travail

La méthodologie retenue repose sur une démarche progressive de génie logiciel : analyse du besoin et définition du périmètre, modélisation fonctionnelle, conception, réalisation, tests et validation. Cet enchaînement relie chaque étape aux besoins identifiés et permet de contrôler progressivement la cohérence du projet.

La durée d'un mois impose de privilégier la simplicité, une progression contrôlée et des validations régulières. Le périmètre de la première version a donc été limité à un ensemble cohérent de fonctions pouvant être analysées, réalisées et vérifiées pendant le stage. Le tableau 1 présente le planning prévisionnel retenu et sa répartition en quatre périodes.

TABLE 1 – Planning prévisionnel du stage

Activité proposée	**S1** 01–07	**S2** 08–14	**S3** 15–21	**S4** 22–31
Compréhension du contexte de l'entreprise	■			
Analyse du besoin et définition du périmètre	■			
Analyse fonctionnelle et UML		■		
Conception de l'architecture et des données		■		
Réalisation backend et frontend			■	
Intégration progressive et finalisation			■	■
Tests et validation				■
Intégration de l'assistance IA et corrections finales				■

Selon ce planning, la première semaine est consacrée au contexte et au cadrage, la deuxième à l'analyse et à la conception, et la troisième principalement à la réalisation et au début de l'intégration. La dernière semaine permet de finaliser la solution, de conduire les vérifications et d'intégrer l'assistance IA ainsi que les corrections finales.

Cette répartition organise les activités du stage sur quatre périodes au cours du mois.

Le périmètre retenu structure le traitement et le suivi des demandes techniques tout en laissant les décisions importantes aux utilisateurs concernés. Le chapitre suivant présente l'analyse des besoins ainsi que la conception fonctionnelle et technique.

Chapitre 2

Analyse et conception

La conception transforme les besoins du stage en responsabilités, règles et structures cohérentes. Elle part des utilisateurs internes et du cycle de vie d'une demande, puis établit l'architecture, les mécanismes de sécurité, l'intégration de l'assistance IA et le modèle de données. Chaque choix est ainsi relié au problème qu'il doit résoudre.

2.1 Spécification des besoins

Acteurs et responsabilités. L'application est réservée aux utilisateurs internes. Le Responsable technique pilote la demande depuis son enregistrement jusqu'à la décision de clôture. L'Agent technique prend en charge les demandes qui lui sont affectées et documente leur résolution. L'Administrateur maintient les comptes et leurs rôles. Cette répartition sépare le pilotage, l'exécution et l'administration, tout en autorisant une même personne à cumuler plusieurs rôles lorsque l'organisation l'exige.

Le Client n'est pas un utilisateur de l'application. Il constitue une entité métier associée à une demande. Cette distinction est importante : elle permet de conserver les informations nécessaires au suivi sans introduire dans la V1 un portail, une authentification ou des droits d'accès pour les clients.

La figure 2 répond à une première question de cadrage : quels acteurs internes utilisent quelles capacités du système ?

Le diagramme complète cette description en visualisant les interactions. Il montre notamment que la connexion est commune aux acteurs, alors que la qualification et le déclenchement de l'assistance IA restent sous la responsabilité du Responsable technique.

Besoins fonctionnels. Les attentes sont regroupées en grandes capacités : contrôler l'accès interne, enregistrer et suivre les demandes, encadrer leur cycle de traitement, administrer les utilisateurs et assister la qualification. Le tableau 2 en donne une synthèse sans reprendre le détail des responsabilités déjà présenté.

Ces besoins décrivent ce que le système doit permettre, et non la manière de le réaliser. Par exemple, « réserver l'accès aux utilisateurs autorisés » est un besoin de protection ; le JWT et le contrôle d'accès basé sur les rôles sont des moyens techniques étudiés plus loin. De même, «

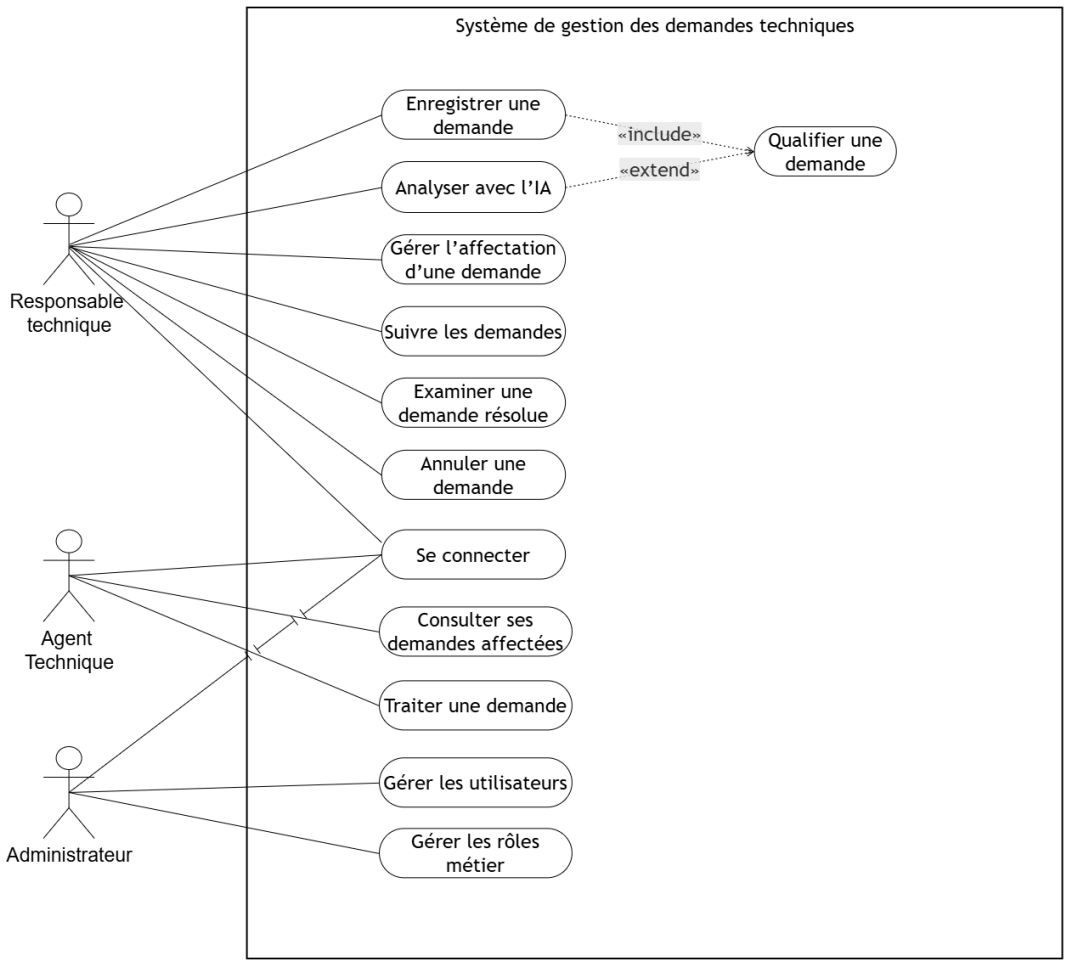


FIGURE 2 – Diagramme global des cas d'utilisation de la V1

conserver les changements importants » est un besoin de traçabilité ; l'entité d'historique et la transaction sont des choix de conception.

Besoins non fonctionnels. La sécurité est nécessaire parce que les actions dépendent du rôle et parfois de la demande concernée. La cohérence garantit qu'une transition et sa trace historique sont enregistrées ensemble. La traçabilité permet ensuite de comprendre les changements importants sans réécrire un événement passé.

La maintenabilité repose sur des responsabilités et des contrats d'échange clairement séparés. L'ergonomie et l'adaptation aux différentes tailles d'écran doivent permettre d'identifier rapidement l'état d'une demande. Enfin, l'indisponibilité du fournisseur IA ne doit jamais empêcher une qualification manuelle.

TABLE 2 – Capacités fonctionnelles de la V1

Capacité	Comportement attendu
Accès interne	Authentifier un utilisateur et orienter son accès selon ses rôles.
Gestion des demandes	Enregistrer, consulter, filtrer et suivre les demandes techniques.
Qualification	Définir une catégorie et une priorité validées par le Responsable technique.
Workflow	Affecter, traiter, résoudre, clôturer, refuser ou annuler selon l'état courant.
Administration	Gérer les comptes, l'état actif et les rôles métier autorisés.
Assistance IA	Proposer une catégorie et une priorité, sans décision automatique.

2.2 Conception fonctionnelle

Cycle de vie. Une demande suit un cycle contrôlé afin qu'aucune action ne contourne les responsabilités définies. Elle passe de `NOUVELLE` à `ASSIGNEE`, puis à `EN_COURS` lorsque l'Agent commence effectivement son travail. Après saisie de la solution, elle devient `RESOLUE`, mais seule la décision du Responsable technique peut la rendre `CLOTUREE`. Une annulation reste possible avant la clôture, avec un motif obligatoire ; `CLOTUREE` et `ANNULEE` sont des états terminaux.

La réaffectation protège la responsabilité du nouvel Agent. Si le traitement avait déjà commencé, la demande revient de `EN_COURS` à `ASSIGNEE` : le nouvel Agent doit confirmer explicitement sa prise en charge. Les informations déjà saisies sont conservées afin de ne pas perdre le travail antérieur.

Enregistrement et qualification. Une demande ne peut être créée qu'avec un client identifiable, une catégorie et une priorité validées. Le Responsable technique peut sélectionner un client existant ou fournir les informations minimales d'un nouveau client. La figure 3 porte le détail de ces deux alternatives et de leur transaction finale.

Résolution. La déclaration de résolution par l'Agent ne clôt pas automatiquement la demande. Le Responsable technique examine le résultat ; s'il le refuse, la demande revient à `EN_COURS`, car un travail complémentaire est attendu. La date de résolution est alors réinitialisée et la décision reste tracée dans l'historique.

Assistance IA. L'analyse est déclenchée explicitement à partir du titre et de la description. Elle produit une proposition modifiable de catégorie et de priorité, jamais une décision automatique. Le Responsable technique conserve donc la responsabilité finale, et une indisponibilité du fournisseur n'interrompt pas la qualification manuelle.

La figure 3 montre comment ces règles sont appliquées lors de l'enregistrement, notamment pour le choix entre un client existant et un nouveau client.

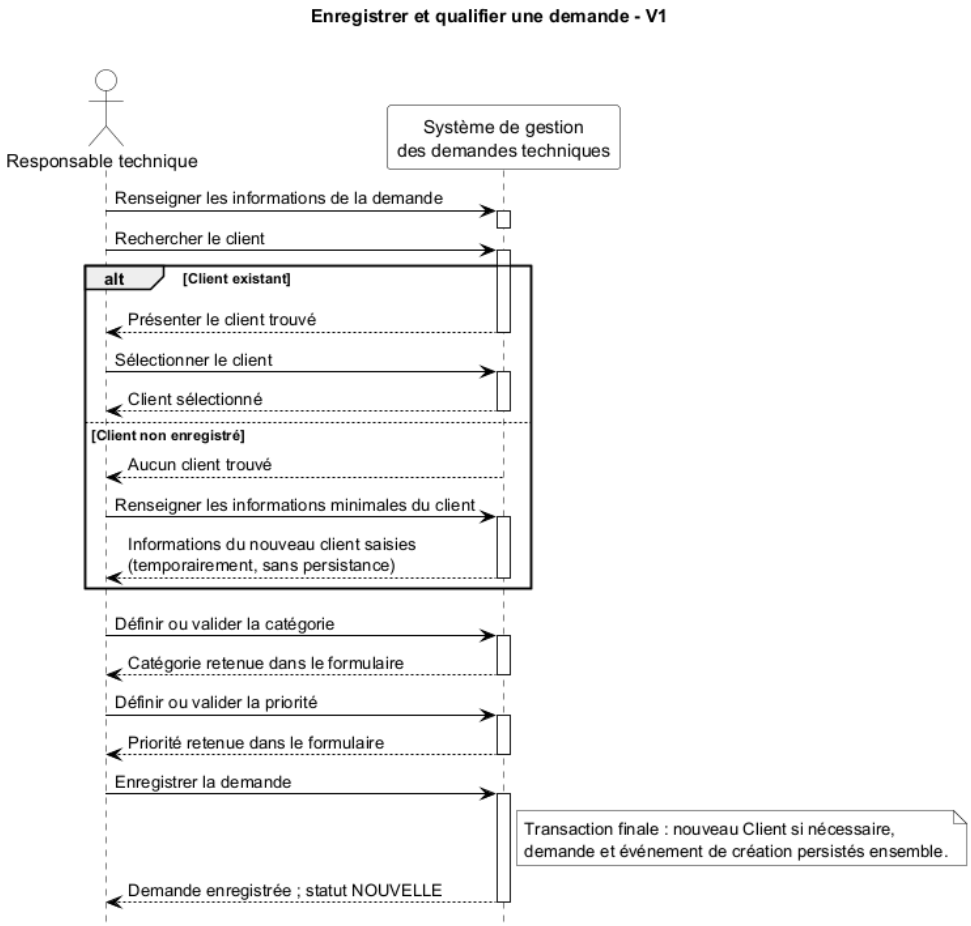


FIGURE 3 – Enregistrement et qualification des demandes

La qualification précède l'enregistrement définitif. Les informations d'un nouveau client, la demande et l'événement de création sont persistés ensemble lors de la transaction finale ; aucune demande partielle n'est créée si les informations requises ne sont pas réunies.

2.3 Conception technique

De la contrainte au choix architectural. Le projet doit fournir une interface web, centraliser les règles métier, conserver des données relationnelles et communiquer avec un fournisseur IA externe. L'association d'Angular, d'une API REST Spring Boot et de PostgreSQL sépare l'interface, le traitement serveur et le stockage. Le backend reste un monolithe modulaire : il conserve un déploiement simple pour la V1 tout en isolant la gestion des demandes, l'identité et l'administration, la sécurité et l'IA.

Le principal problème de conception est d'éviter le mélange entre présentation, traitement HTTP, règles métier et persistance. Dans Angular, les fonctionnalités sont regroupées par

domaine et les services d'accès à l'API évitent de placer les règles métier dans les composants. Dans Spring Boot, les contrôleurs reçoivent les requêtes HTTP, les services orchestrent les cas d'utilisation, le domaine protège les invariants et les repositories isolent la persistance. Les objets de transfert de données (DTO) empêchent enfin d'exposer directement les entités persistantes.

Cette séparation rend les responsabilités plus lisibles et facilite l'évolution de chaque couche. Elle illustre le principe de responsabilité unique (SRP), dans la mesure où chaque composant possède un rôle principal identifiable. Cette application reste ciblée : elle ne permet pas d'affirmer que toutes les classes respectent parfaitement ce principe ni que l'ensemble des principes SOLID est appliqué uniformément.

L'intégration IA pose un autre problème : le cœur métier ne doit pas dépendre directement d'un fournisseur externe. Pour le résoudre, l'abstraction `AnalyseIaPort` définit le besoin d'analyse, tandis que `GroqAnalyseIaAdapter` réalise l'appel au fournisseur. Ce choix illustre le principe d'inversion des dépendances (DIP) et l'organisation Port/Adapter ; il isole le fournisseur et limite l'impact d'un changement d'intégration.

Le Repository Pattern répond de la même manière au besoin d'isoler la persistance des services métier. Les DTO assurent, quant à eux, le découplage entre les contrats HTTP et les entités persistantes. Ils sont utilisés ici comme une technique de séparation des échanges, sans être présentés comme une solution universelle.

Le State Pattern n'a pas été retenu pour le workflow. Pour la V1, un enum de statut et des opérations métier explicites suffisent à vérifier les transitions. Cette décision réduit la complexité tant que le nombre d'états et de règles reste limité ; elle pourrait être réévaluée si le cycle de vie s'étend fortement.

Sécurité conçue. Le besoin de protection impose d'abord de distinguer l'authentification de l'autorisation. L'authentification vérifie l'identité et l'état actif du compte. Après cette vérification, un JSON Web Token (JWT) de durée limitée est transmis avec les requêtes protégées (Jones, Bradley et Sakimura 2015).

L'autorisation détermine ensuite les actions permises. Le contrôle d'accès basé sur les rôles (RBAC) attribue les permissions générales aux rôles Responsable technique, Agent technique et Administrateur. Il est complété par des contrôles contextuels : un Agent ne peut traiter que les demandes qui lui sont affectées, chaque transition dépend de l'état courant et un Administrateur ne peut pas se désactiver lui-même.

Les mots de passe sont hachés avec `PasswordEncoder` et les secrets sont fournis par une configuration externe. Angular restaure la session depuis `sessionStorage` pour préserver l'expérience utilisateur après un rechargement. Cette restauration locale ne constitue cependant pas une autorisation : le backend vérifie la signature et l'expiration du JWT, relit l'état actif et les rôles courants, puis contrôle l'action demandée. L'API reste ainsi l'autorité de sécurité.

Assistance IA. Le titre et la description sont transmis au service d'analyse, qui renvoie une proposition structurée de catégorie et de priorité. Le port d'intégration permet de contrôler cette

réponse et d'isoler Groq du cœur métier. La qualification manuelle demeure disponible lorsque le système externe est indisponible, et toute proposition reste soumise à une validation humaine.

La figure 4 situe les grandes frontières de la solution et répond à la question : quels composants collaborent pour fournir les fonctions métier ?

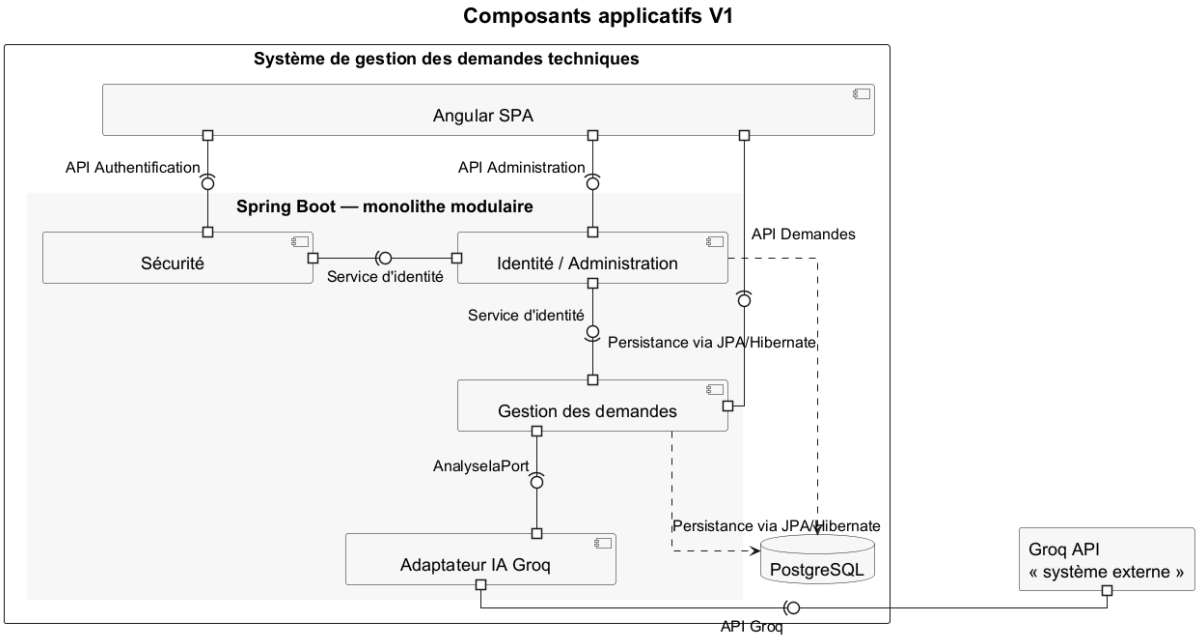


FIGURE 4 – Composants applicatifs et systèmes externes de la V1

Le diagramme distingue l'application Angular, le monolithe modulaire Spring Boot, PostgreSQL et l'API Groq. Il rend aussi visible l'isolation de l'intégration IA derrière un port, tandis que les modules de gestion des demandes et d'identité accèdent à PostgreSQL par la persistance JPA/Hibernate.

La figure 5 affine cette vue en montrant comment les responsabilités sont réparties à l'intérieur du backend.

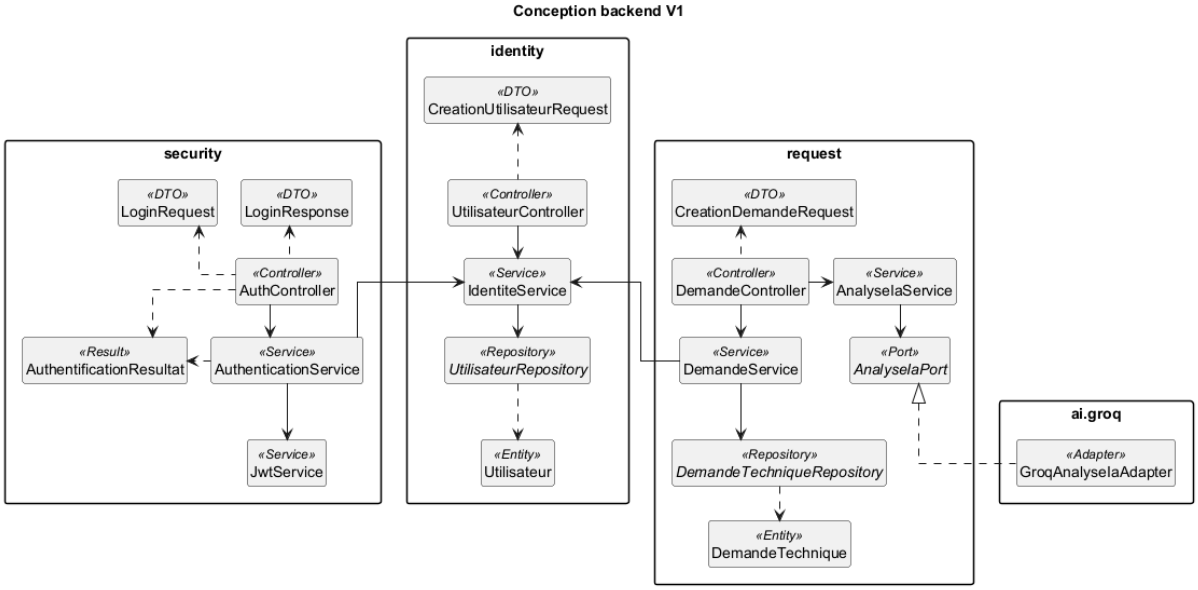


FIGURE 5 – Organisation des responsabilités dans la conception backend

Les contrôleurs portent la frontière HTTP, les services orchestrent les cas d'utilisation et les repositories isolent la persistance. Le couple `AnalyseIaPort/GroqAnalyseIaAdapter` matérialise la dépendance du cœur applicatif vers une abstraction, puis l'implémentation de cette abstraction par l'adaptateur externe. Les deux figures sont donc complémentaires : la première expose les composants, la seconde les rôles techniques qui structurent le backend.

2.4 Conception de la base de données

La base relationnelle doit préserver trois dimensions complémentaires : l'identité des utilisateurs, le contenu et le cycle des demandes, puis la trace des événements. Le modèle répartit ces responsabilités entre des structures distinctes afin de protéger leurs contraintes et d'éviter la concentration de toutes les informations dans une seule table.

La figure 6 présente les entités persistantes et répond à la question : quelles données sont conservées et selon quelles cardinalités ?

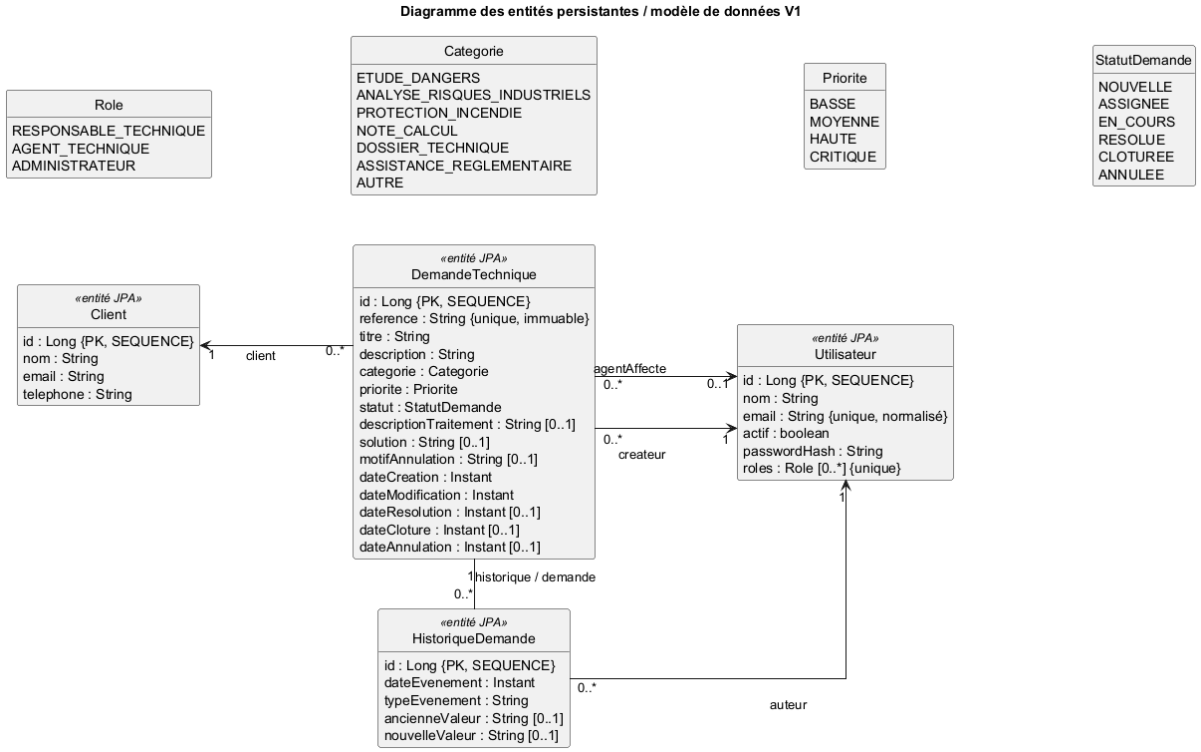


FIGURE 6 – Entités persistantes et cardinalités du modèle de données V1

Le modèle place la demande au centre des relations. Chaque demande possède un client et un créateur, peut être affectée à un Agent et regroupe plusieurs événements d'historique. Les énumérations encadrent les rôles, catégories, priorités et statuts autorisés sans les transformer en entités indépendantes.

L'identité repose sur Utilisateur et ses rôles. L'adresse électronique, obligatoire et unique, identifie le compte ; l'état actif permet d'en bloquer l'usage sans supprimer son historique. Une table d'association conserve les rôles cumulables attribués à chaque utilisateur.

DemandeTechnique constitue le cœur du modèle. Elle référence obligatoirement le Client concerné et son créateur, possède une référence métier unique et peut être affectée à un seul Agent. Les catégories, priorités et statuts sont persistés sous forme de codes textuels afin de conserver des valeurs explicites. L'adresse électronique d'un Client n'est pas unique, car aucune règle métier validée ne l'impose.

HistoriqueDemande assure la traçabilité. Chaque événement est rattaché à une demande et à son auteur, avec les anciennes et nouvelles valeurs lorsqu'elles sont utiles. Une correction crée un nouvel événement au lieu de réécrire le passé. Les dates métier sont représentées par des instants et stockées en `TIMESTAMP WITH TIME ZONE`, en cohérence avec le type `Instant` côté Java (PostgreSQL Global Development Group 2026).

Le dictionnaire complète le diagramme en précisant les attributs, leurs types et les contraintes qui ne sont pas détaillées graphiquement.

En résumé, la conception relie chaque besoin à une responsabilité identifiable : les cas d'utilisation structurent le comportement, l'architecture sépare les traitements, la sécurité protège les opérations et le modèle de données garantit les relations essentielles.

TABLE 3 – Dictionnaire de données synthétique

Entité	Attribut	Type	Contrainte et signification
Utilisateur	id	Long / BIGINT	Clé primaire, séquence.
Utilisateur	email	String / VARCHAR	Obligatoire, normalisé et unique.
Utilisateur	actif	boolean / BOOLEAN	Obligatoire ; compte autorisé ou désactivé.
Utilisateur	roles	Set<Role>	Association ; plusieurs rôles possibles.
Client	id	Long / BIGINT	Clé primaire, séquence.
Client	nom, email, telephone	String / VARCHAR	Trois informations obligatoires ; email non unique.
DemandeTechnique	reference	String / VARCHAR	Obligatoire, unique et immuable ; référence métier.
DemandeTechnique	titre, description	String / TEXT	Obligatoires ; contenu de la demande.
DemandeTechnique	categorie	Categorie / VARCHAR	Obligatoire ; valeur de la liste V1.
DemandeTechnique	priorite	Priorite / VARCHAR	Obligatoire ; BASSE à CRITIQUE.
DemandeTechnique	statut	Statut / VARCHAR	Obligatoire ; état du cycle de vie.
DemandeTechnique	agentAffecte	Utilisateur / FK	Optionnel ; au plus un Agent affecté.
DemandeTechnique	dates métier	Instant / timestamptz	Création et modification obligatoires ; résolution, clôture et annulation optionnelles.
HistoriqueDemande	typeEvenement	String / VARCHAR	Obligatoire ; nature du changement.
HistoriqueDemande	ancienne/nouvelle valeur	String / TEXT	Optionnelles ; valeurs avant et après.
HistoriqueDemande	demande, auteur	FK	Obligatoires ; origine et responsable de l'événement.

Chapitre 3

Réalisation et validation

3.1 Vue d'ensemble de la solution réalisée

Angular fournit l'application web et les parcours utilisateur. Spring Boot expose l'API et applique les règles métier. PostgreSQL conserve les données et l'historique. Cette répartition permet de faire évoluer l'interface sans déplacer les invariants métier dans le navigateur.

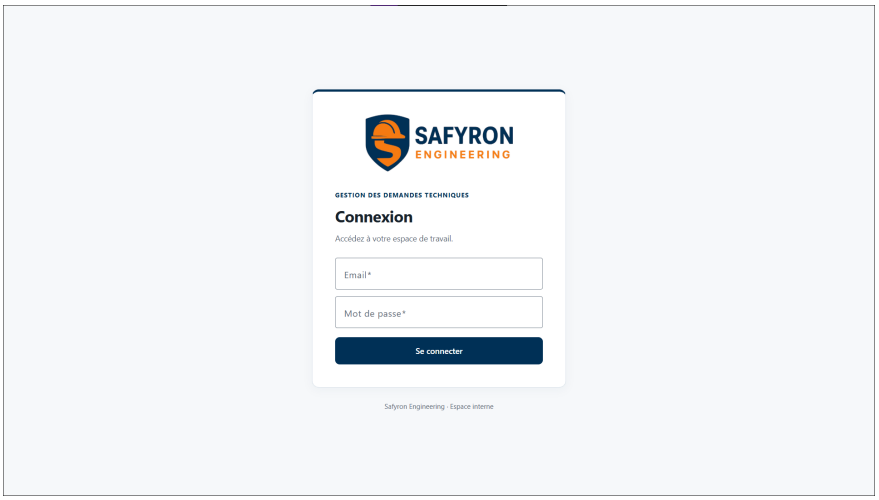


FIGURE 7 – Connexion à l'application

Après authentification, le frontend conserve le token et le snapshot minimal utile à l'orientation de l'interface. Les autorisations effectives restent décidées par l'API.

3.2 Parcours complet d'une demande

Le parcours commence par la création et la qualification. Le Responsable technique renseigne un titre, une description, un client, une catégorie et une priorité. Il peut demander une suggestion IA, mais garde la possibilité de modifier les valeurs avant l'enregistrement.

La liste permet ensuite de retrouver les demandes par filtres, tri et pagination côté serveur. Après affectation, l'Agent traite la demande, renseigne la solution et déclare la résolution. Le

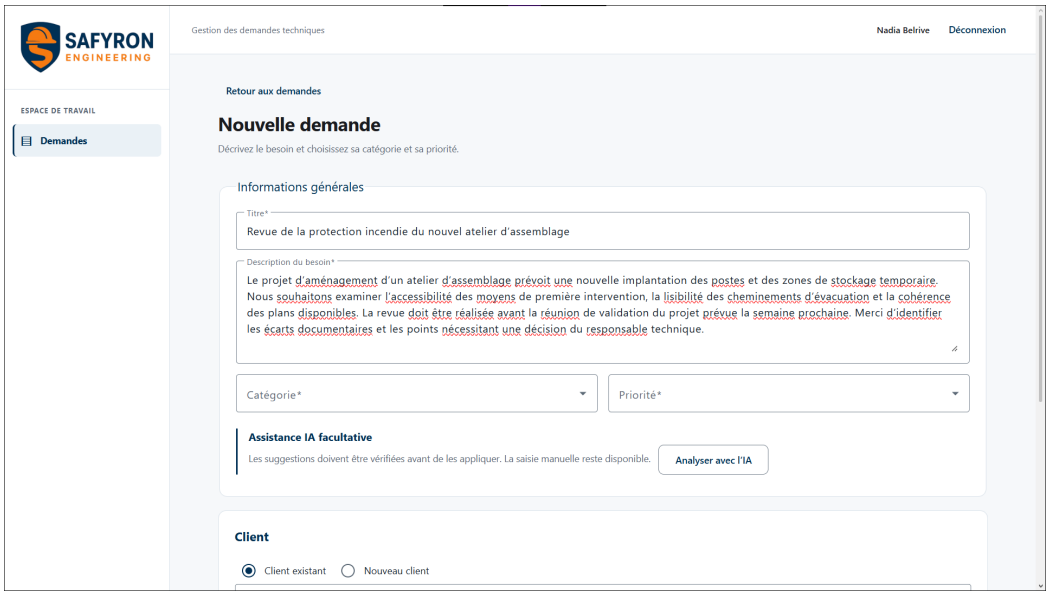


FIGURE 8 – Création et qualification initiale d'une demande

Responsable examine cette proposition : il clôture la demande ou la refuse pour demander une reprise.

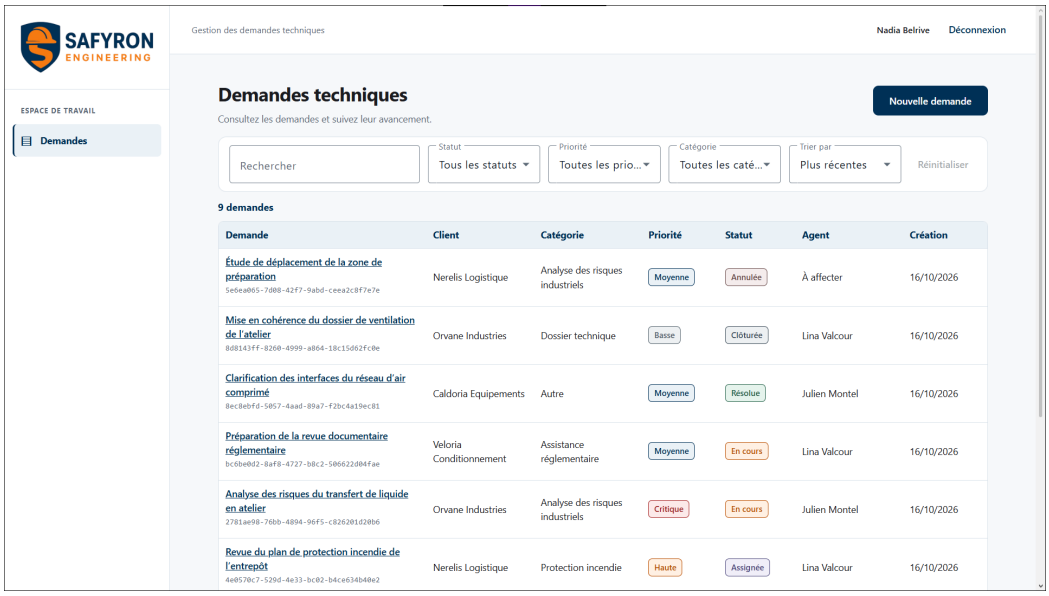


FIGURE 9 – Liste filtrable des demandes

Ces écrans illustrent une propriété importante : l'interface présente des actions dépendantes du rôle et du statut, mais cette présentation ne remplace pas le contrôle côté serveur.

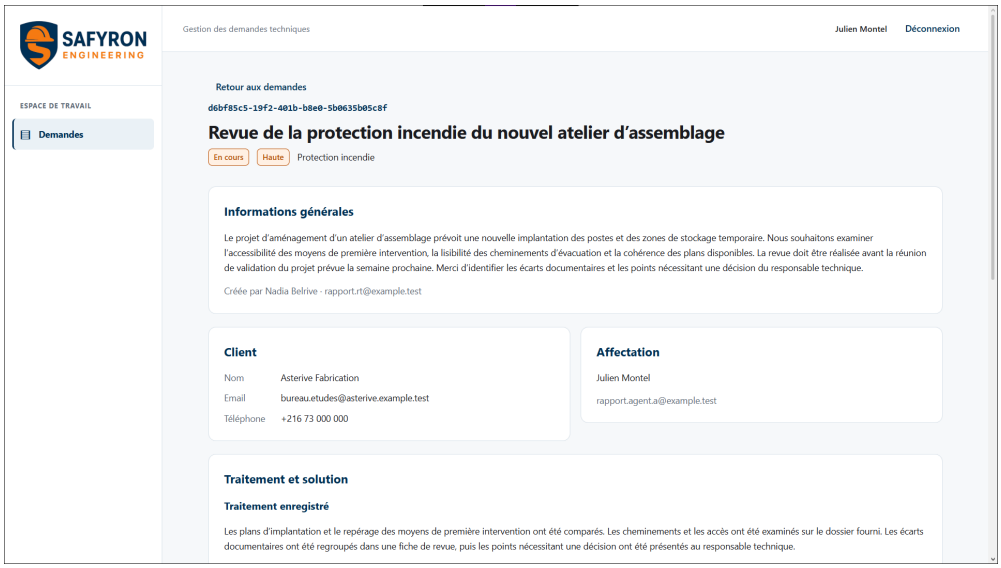


FIGURE 10 – Traitement d'une demande affectée

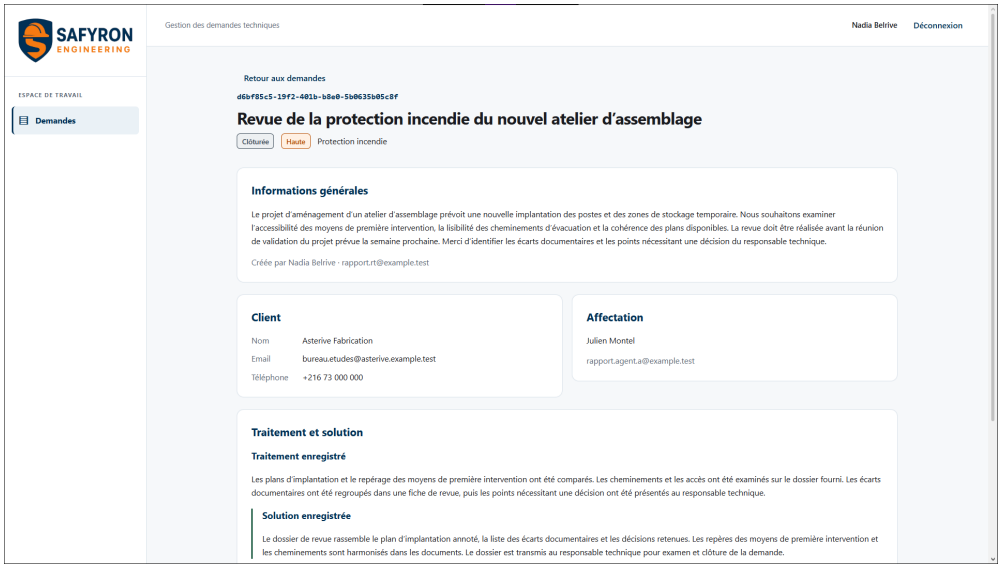


FIGURE 11 – Clôture après examen de la résolution

3.3 Administration et contrôle des accès

L'administration permet de créer et modifier les comptes, de gérer leur état actif et de remplacer leurs rôles métier. L'auto-désactivation du compte courant est refusée par l'API avec une réponse 409 et l'interface conserve un retour compréhensible.

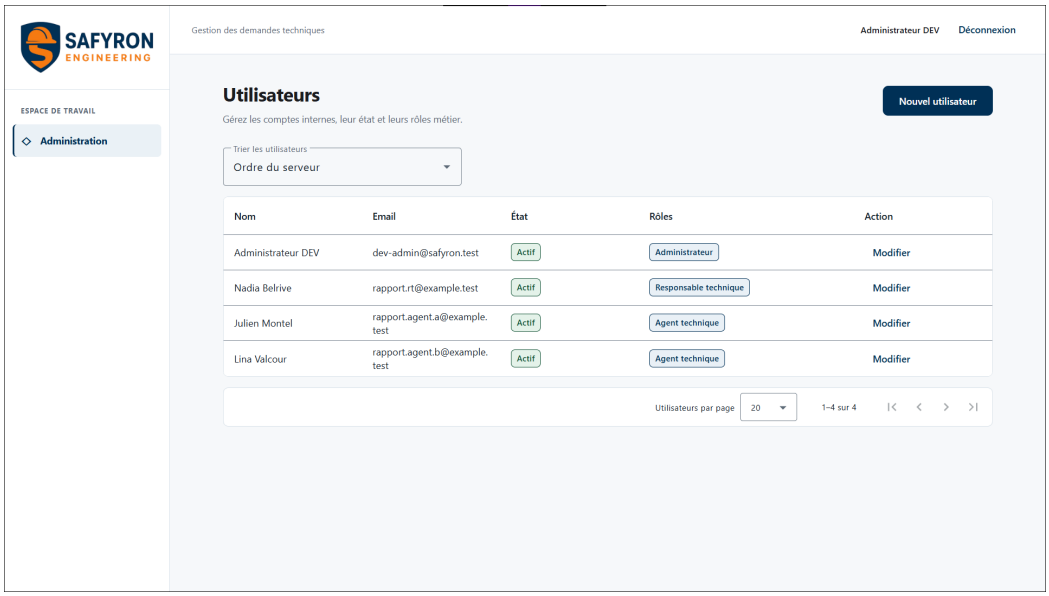


FIGURE 12 – Gestion des utilisateurs internes

Le backend relit l'utilisateur et ses rôles à chaque requête protégée. Un rôle supprimé ou un compte désactivé ne conserve donc pas ses droits simplement parce qu'un snapshot ancien existe dans le navigateur.

3.4 Assistance IA et validation humaine

L'analyse IA est déclenchée depuis le formulaire de nouvelle demande. Le frontend envoie uniquement le titre et la description à l'endpoint backend. Le serveur appelle l'adaptateur Groq via ‘AnalyseIaPort‘, contrôle la réponse puis renvoie une proposition temporaire. La catégorie et la priorité ne sont jamais appliquées automatiquement.

Une modification du titre ou de la description invalide la suggestion. Lorsque la clé Groq est absente, l'API renvoie une indisponibilité contrôlée et le formulaire reste utilisable manuellement. Le benchmark exploratoire porte sur 24 exemples : sorties valides dans les 24 cas, catégorie correcte dans 23 cas et priorité dans 22 cas. Ces valeurs ne permettent pas d'annoncer une performance générale ni une supériorité universelle d'un fournisseur.

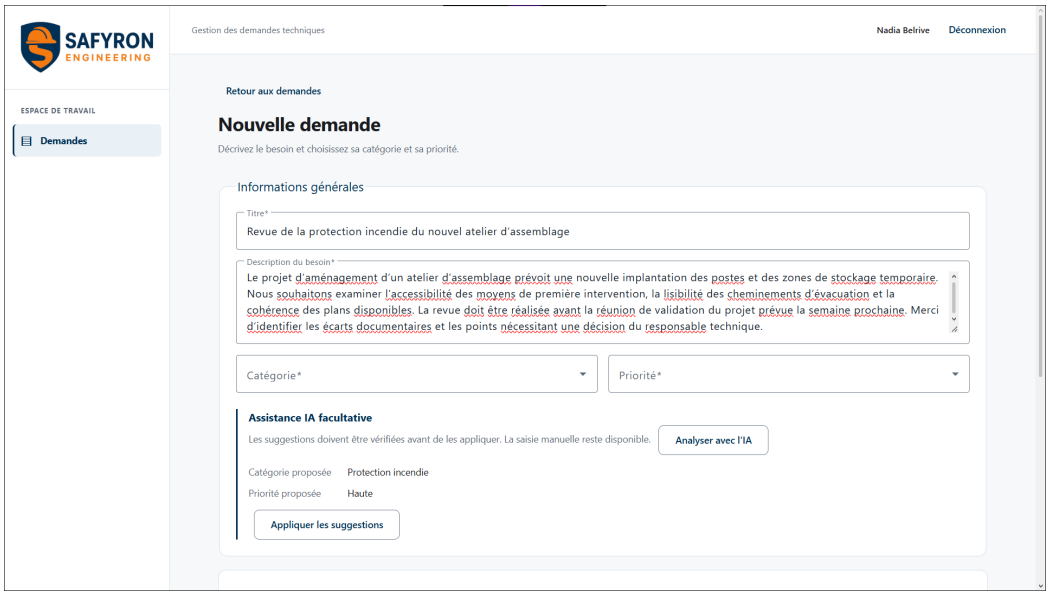


FIGURE 13 – Assistance IA lors de la qualification

3.5 Tests et validation

La validation backend atteint 270 tests réussis avec ‘clean test‘ et ‘clean verify‘, sans échec, erreur ou test ignoré. Les tests IA hors ligne utilisent H2 et un stub HTTP local ; ils ne constituent pas un appel Groq. Le frontend a été vérifié par ‘npm ci‘, ‘npm run build‘ et ‘npm test – –watch=false‘, avec 184 tests réussis dans 15 fichiers pour la clôture IA.

La validation fonctionnelle humaine a couvert la connexion, la liste, la création, la sélection client, les transitions de demande, l’administration, l’auto-désactivation et l’assistance IA réelle en local. Une revue responsive et accessibilité a également été effectuée sur plusieurs tailles. Ces observations ne remplacent pas une campagne de test de charge ou une validation de production.

3.6 Limites et perspectives

Le rapport ne prétend pas mesurer la performance en production, la résistance à la charge ou la qualité IA sur un corpus représentatif du long terme. Le périmètre reste interne et le client n’interagit pas directement avec la plateforme. La session navigateur utilise ‘sessionStorage‘, ce qui permet une restauration après rechargement mais conserve les risques propres à un stockage accessible au JavaScript du navigateur.

Les perspectives cohérentes avec la V1 sont l’élargissement du corpus d’évaluation IA, l’ajout de mesures répétées et l’amélioration de l’observabilité. Toute extension du workflow, du portail client ou du déploiement devra être précédée d’une analyse de besoin et d’une validation dédiée.

TABLE 4 – Niveaux de validation et portée

Validation	**Ce qu'elle permet d'affirmer**
Tests backend	Les comportements couverts par 270 tests réussissent dans l'environnement vérifié.
Tests frontend	La construction et les scénarios automatisés couverts réussissent.
Intégration locale IA	Le parcours Angular vers Spring Boot et Groq a fonctionné dans les conditions observées.
Validation humaine	Les parcours et l'ergonomie vérifiés ne présentent pas de blocage constaté dans le périmètre revu.
Benchmark IA	Résultats exploratoires sur 24 exemples, non généralisables.

Conclusion générale

Le stage a conduit à une application V1 couvrant l'enregistrement, la qualification, l'affectation, le traitement et le suivi des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés et les validations humaines montrent que le périmètre V1 est fonctionnel dans les conditions vérifiées.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. Des travaux ultérieurs pourraient élargir le corpus d'évaluation, renforcer l'observabilité et étudier les besoins de déploiement, sous réserve de validations supplémentaires.

Références

Jones, Michael, John Bradley et Nat Sakimura (mai 2015). *JSON Web Token (JWT)*. url : https://www.rfc-editor.org/rfc/rfc7519.html (visité le 11/10/2026).

PostgreSQL Global Development Group (2026). *Date/Time Types*. url : https://www.postgresql.org/docs/current/datatype-datetime.html (visité le 11/10/2026).